

On the Use of ECDSA with Hierarchical Public Key Delegation in Identity-based Scenarios

Lucas C. Cardoso  and Marcos A. Simplicio Junior 

University of São Paulo, São Paulo, Brazil

Abstract. In 2009, Galindo and Garcia proposed the usage of concatenated Schnorr signatures for the hierarchical delegation of public keys, creating a quite efficient identity-based signature scheme (IBS). Essentially, the scheme builds upon the Schnorr signature scheme to generate a primary signature, part of which is then used as a secret key to produce signatures on subsequent messages. The resulting IBS is proven secure against existential forgery on adaptive chosen-message and adaptive identity attacks using variants of the Forking Lemma. In this paper, our goal is to answer the following question: would it be feasible to build upon the widely used elliptic curve digital signature algorithm (ECDSA) scheme to obtain a similarly secure and efficient IBS? We answer this affirmatively, opening interesting possibilities not only for identity-based signatures with ECDSA but also for applications such as secure credential delegation. This latter application is of particular interest considering the wide support for ECDSA in web- and cloud-oriented authentication systems (e.g., based on JSON Web Tokens). The resulting scheme is proven secure, combining the Bijective Random Oracle model and the existential unforgeability game in an identity-based setup. Our results show that even considering ECDSA’s non-linear characteristic and more convoluted verification process when compared to Schnorr signatures, it is possible to obtain shorter signatures than Galindo-Garcia’s scheme.

Keywords: ECDSA · identity-based signatures · secure credential delegation · security proof

1 Introduction

Elliptic curve cryptography’s popularity is due to its advantages compared to RSA [RSA78] and pairing-based schemes, since it allows faster computation while providing smaller keys and shorter signatures [LV01]. The elliptic curve digital signature algorithm (ECDSA) [Nat23] and its close relative DSA were proposed during the 1990s and first adopted by NIST in early 2000s. ECDSA ensures security properties such as data integrity, authenticity, and non-repudiation, and is broadly used in modern systems. Examples include cryptocurrencies (e.g., *Bitcoin* [Nak08] and *Ethereum* [Eth24]), network protocols (e.g., Transport Layer Security [Res18]), etc.

Since its proposal approximately thirty years ago, ECDSA’s security analysis has been approached from various angles. Numerous attacks based on nonce leakage [BH19, GKL⁺20, ANT⁺20, XDWL23, GWHH25] demonstrate that the usage of ECDSA in modern computing should be done carefully since it is very prone to side-channel attacks. From a theoretical point of view, previous analyses were first made to study the hardness of the discrete logarithm problem under the Generic Group Model [Nec94, Sho97, BPVY00]. Later, the security of ECDSA was proven in the generic group model under strong idealization around the conversion function f , which maps a point on the curve to its x -coordinate

E-mail: lcardoso@larc.usp.br (Lucas C. Cardoso), mjunior@larc.usp.br (Marcos A. Simplicio Junior)

modulo a prime number, by Brown in 2002 [Bro02], followed by related works published in 2005 [Bro05]. However, Stern et al. [SPMLS02] noted that the analysis performed by Brown led to strong unforgeability, a clear distortion since ECDSA signatures are known to be malleable, i.e., if $\sigma = (s, t)$ results a valid signature over message m , then so does $\sigma' = (-s, t)$. Up to this moment in history, pure ECDSA's analysis was based on some strong idealizations rather than more concrete assumptions, thus representing an interesting question of whether a more practical result could be achieved.

Related Works. The security analysis of ECDSA under the generic group model endured as the main result until the works produced by [FKP16, FKP17]. Making use of both a non-idealized group and a conversion function that is only partially idealized, the authors concluded the security of ECDSA under adaptive chosen-message attacks in the discrete-logarithm setting under the bijective random oracle model in a framework named GENDSA modeling f . Recently, work by [HK23] focused on studying the programmability of the conversion function under the bijective random oracle model. Their result showed that provable security requires programmability of (part of) the conversion function to reduce existential unforgeability under adaptive chosen-message attack game to the discrete logarithm problem. This new perspective on the projection function was pivotal to the development and analysis of other ECDSA-based schemes.

In face of these new analyses, numerous schemes followed. Qin et al. [QCY21] discusses a new ECDSA-based blind signature scheme proven secure under the algebraic variant of the bijective random oracle, Lindell [Lin21] builds an efficient two-party ECDSA-based threshold scheme proven secure under a new security assumption, and Groth and Shoup [GS22] add key derivation and presignatures mechanisms to ECDSA, making the new scheme suited for threshold setting. Recently, Cho et al. [CFOS25] discuss a new non-interactive non-falsifiable assumption called the *Circular Discrete Logarithm Assumption* used to prove the security of not only the ECDSA scheme, but also Schnorr [Sch89], and Sparkle+ [CKM23]. While these results represent the conditions needed to achieve provable security of ECDSA and rely on (still, even though somewhat partially) idealized models, they represent an alternative tool to evaluate the security of ECDSA-like schemes applied to different scenarios.

One of these scenarios is the identity-based one. Identity-based cryptosystems were first proposed in [Sha85] and represent major advances in PKI requirements. In an identity-based setting, the user's public key can be (quite simply) his identity, i.e., his name, network address, email, etc. This represents a major implication when deriving identity-based signatures, because, in such cases, after sending his/her identity, a *key generation center* – KGC – issues a secret key, thus providing a higher efficiency when compared to certificate-based schemes, while adding more responsibility to the issuing authority.

In 2009, Galindo and Garcia proposed a Schnorr-like identity-based signature, where the private key represents one of the practical benefits in resource-constrained environments, as it allows for a certificateless setup in which public keys are delegated to an entity, thus enabling it to verify the validity of a signature [GG09]. This is accomplished by concatenating Schnorr signatures, a well-known technique studied since the early 1990s [Gir91] with applications in modern cryptographic schemes [MPSW19].

Also, as mentioned by [BZ23], the Galindo-Garcia scheme also entails possible usage of one-time signatures as succinct non-interactive arguments of knowledge, i.e., a party can prove that he/she possesses a valid signature over a first message by using part of the signature as a private key to sign a second message. This is also known as a Proof of Knowledge (PoK) of the signature, a direct consequence of the Sigma protocol's *special soundness* property, and extremely useful to build authentication protocols. The construction of Σ -protocols has been a recurrent topic of study as an effort to leverage properties that would

benefit authentication systems supporting ECDSA signatures [AGM18, FHL22, CLR24]. Woo et al. [WOS⁺25] try a different approach, using a two-step setup: *i*) they encode the ECDSA verification algorithm as a rank-one constraint system (R1CS) [BSCR⁺19] to further *ii*) use a zkSNARK to prove knowledge of a solution [BCCT12].

Considering recent contributions to ECDSA analysis and the similarities with Schnorr signature scheme, our work’s target is to answer the question: *would it be feasible to build an IBS with hierarchical public key delegation using an ECDSA signature as the user’s secret key, thus allowing ECDSA’s signature concatenation?*

Contributions. In this paper, our contribution is threefold:

- i*) *An ECDSA-like IBS*: to the best of our knowledge, we present the first ECDSA-like identity-based signature scheme allowing ECDSA’s signature concatenation. We utilize the idea proposed in Galindo and Garcia’s scheme, with a slight modification: in addition to using an ECDSA signature as the user’s private key, we also leverage the cyclic nature of the elliptic curve to utilize the t part of an ECDSA signature $\sigma = (s, t)$ to define the generator that will be used during the signing algorithm of our IBS. This opens interesting possibilities in authentication systems with native support for ECDSA signatures, especially web and cloud-oriented ones.
- ii*) *Security proof in the bijective random oracle model*: we prove the security of our model under the *discrete logarithm assumption* over elliptic curves under the *bijective random oracle* using GENDSA’s framework proposed by Fersch et al. and conclude *existential unforgeability under adaptive chosen-message and adaptive identity attack* security in two steps: first, we reduce existential unforgeability under chosen-message and adaptive identity attack to unforgeability under key-only and identity attack to further reduce it to ECDSA’s security under key-only attacks. We also present an alternative result which consists on reducing the unforgeability under key-only and identity attack to the semi-logarithm problem. Figure 1 summarizes contributions *i*) and *ii*);

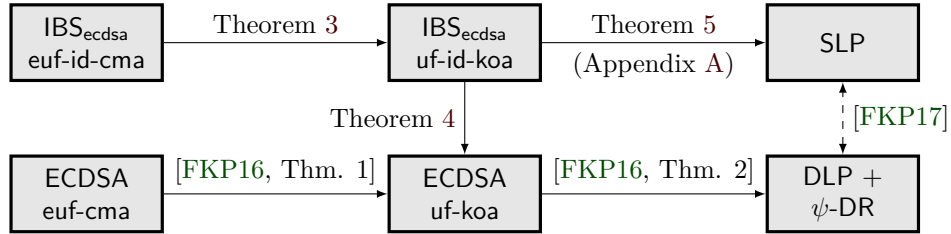


Figure 1: Summary of the security proof under GENDSA’s framework. The dashed line represents an equivalent result.

- iii*) *Comparison between our scheme and Galindo-Garcia’s*: we evaluate the performance between our ECDSA-based scheme and Schnorr-like scheme with hierarchical delegation of public key, and discuss the usage of our scheme in scenarios with native ECDSA support in delegated credentials, e.g., OAuth2 and JSON Web Tokens.

Paper Outline. The rest of the paper is organized as follows. Section 2 defines the notation, the model in which we will study our scheme, and the cryptographic assumptions used through the security analysis. Section 3 discusses the core ideas of the Galindo-Garcia identity-based signature scheme, emphasizing their challenges and obtained results. Section 4 defines the proposed scheme. Section 5 details the proof itself using a game-based strategy. Section 6 analyzes the obtained performance of our scheme when compared to

Galindo-Garcia's and presents applications that could benefit from our scheme. Section 7 concludes the work.

2 Preliminaries

2.1 Notation

SETS. For any finite set S , $x \leftarrow S$ denotes the uniformly random sampling of an element $x \in S$ and $|S|$ is the number of elements. We use the notation $[n]$ to write the numbers from 1 to n and $[m : n]$ to represent the numbers from m to $n \geq m$. Finally, $\{0, 1\}^*$ is the set of all binary strings with arbitrary finite size.

ALGEBRA. Through the paper we write $\mathbb{Z}_p := \mathbb{Z}/p\mathbb{Z}$ as the ring of integers modulo prime number p , and $\mathbb{Z}_p^* := \mathbb{Z}_p \setminus \{0\}$. For an Abelian group $\mathbb{G}$ written additively with neutral element $\mathcal{O}$ (in the case of elliptic curves, this is the point at infinity), we write $\mathbb{G}^* := \mathbb{G}/\{\mathcal{O}\}$. For elliptic curves, let q be a prime number, $E/\mathbb{G}$ be an elliptic curve, $E(\mathbb{F}_q)$ the set of points with coordinates defined over $\mathbb{F}_q \times \mathbb{F}_q$ such that $|E(\mathbb{F}_q)| = p$ indicates the number of elements of this set and $pG = \mathcal{O}$ where G is called the generator of the curve.

SECURITY GAMES. We call $\mathcal{A}$ an attacker on our scheme and write $\mathcal{A}^{O_1, O_2, \dots, O_n}$ when the attacker has access to oracles $O_{i \in [n]}$, while $\mathcal{C}$ represents the challenger. We use $\mathcal{AMS}$ letters $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{F}$ to represent attackers. We use $x \leftarrow \mathcal{A}$ to say that x is the output of a deterministic algorithm $\mathcal{A}$ while $x \xleftarrow{\$} \mathcal{A}$ when the algorithm is probabilistic. The security parameter is represented by λ . The string 1^λ consists of only 1 as a digit and has a length of λ . During the proof, we use code-based security games [BR04, BR06]. A game $\mathbf{G}$ is defined as a probability experiment in which an adversary $\mathcal{A}$ interacts with an implicit challenger $\mathcal{C}$ that answers oracle queries posed by $\mathcal{A}$. More formally, a game is a sequence of programs such that $\mathbf{G} = (\text{Init}, P_1, \dots, P_n, \text{Finalize})$ where $P_{i \in [n]}$ are the *oracles* of the game. The binary output b of the i -th game $\mathbf{G}_i$ between the challenger $\mathcal{C}$ and the attacker $\mathcal{A}$ is denoted by $\mathbf{G}_i^{\mathcal{A}} \Rightarrow b$. The attacker *wins* if $b = 1$. Lastly, we use colored boxes to indicate changes between consecutive games and highlight their differences.

ORACLES AND HASH FUNCTIONS. We use **sans serif fonts** to represent an oracle illustrating some idealized cryptographic primitive, e.g., $\mathcal{O}$. We represent $\#O(x)$ as the index on which the oracle query to input x was made, $|\mathcal{O}|$ as the range of elements in $\mathcal{O}$'s table and $q_{\mathcal{O}}$ as the maximum number of queries allowed. The attacker can make as many queries x_i to $\mathcal{O}(\cdot)$ as he/she wants and obtain $\mathcal{O}(x_i)$ as output. The table associated with the oracle is represented by $\mathfrak{L}$. The entries of this table are defined according to the oracle's arguments. Finally, when dealing with tables, the associated query is represented by enclosed brackets, e.g., $\langle a_i, b_i, \dots, z_i \rangle$ where $a, b, \dots, z$ are the entries of the table. Digests of hash functions are presented by h (or h', h'' , etc, when this is the case).

COLOR SCHEMA. Finally, we use different colors to represent pairs of keys, signatures, points on the curve, and others for conciseness. Public keys (or points on the curve) are represented with upper case letters in **blue**, generators are represented in **orange**, and private keys are represented in **red**. Signatures are written in **brown**. Forgeries are written in **magenta** and a star $*$ (for example σ^* represents a forged signature). Nonces are represented in **green**, digests are written in **cyan**, and commitments in **purple**.

2.2 Modeling the Conversion Function of ECDSA

The conversion function is given by $f : E(\mathbb{F}_q) \mapsto \mathbb{Z}_p$, mapping a point $Q \in E(\mathbb{F}_q)$ to its x -coordinate modulo prime number p , i.e., $f(Q) = x \bmod p$, $Q := (x, y) \in \mathbb{F}_q \times \mathbb{F}_q$ without preserving a visible algebraic structure, meaning that the only connection between its domain and its range seems to be that $|\mathbb{G}| = |\mathbb{Z}_p|$. However, practical instantiations of f are not bijective, thus resulting that $|f(\mathbb{G})| < |\mathbb{Z}_p|$ as highlighted by [FKP16]. Guided

by those pieces of evidence, the most recent analysis of ECDSA relies on modeling f by combining its disruptive algebraic nature and real-world properties. Fersch et al. model f as a composition of functions, i.e., $f(x) = (\psi \circ \Pi \circ \varphi)(x)$.

We use the following notation to represent the (co)domain of functions ψ, Π, φ : $\mathbb{A}$ contains binary strings $\alpha \in \{0, 1\}^\lambda$, $\lambda := \lceil \log_2 q \rceil$ with $2^\lambda \leq q < 2^{\lambda+1}$; $\mathbb{B}$ corresponds to the interval $[2^\lambda - 1]$. Every element of $\mathbb{B}$ is represented by β . Both ψ and φ are fast and efficient functions. Function φ reflects the structure of f that involves only its domain, i.e., it maps a point on the curve onto its representation as a binary string of length λ :

$$\varphi : E(\mathbb{F}_q) \mapsto \mathbb{A} \subseteq \{0, 1\}^\lambda$$

Function ψ reflects the structure of f 's range by mapping a point onto its x -coordinate modulo p . The novelty of this model is to use Π as a bijection to disrupt any algebraic link between the domain and the range of f . In our case, φ is such that any two elliptic curve points P and P' with the same x -component will be mapped to the same element, and function ψ is the *modulo p function* that implements the characteristic bias of the range of f . Note that f is invertible since both φ and ψ are invertible. More formally, φ is a semi-injective function (or 2-to-1).

Definition 1 (Semi-Injective Function, [HK23]). Let $\mathbb{G}$ be a prime order group and $\mathbb{D}$ a set. A function $\varphi : \mathbb{G}^* \rightarrow \mathbb{D}$ is called semi-injective if

- i) its range $\varphi(\mathbb{G}^*) \subseteq \mathbb{D}$ is efficiently decidable and
- ii) it is either injective or a 2-to-1 function, i.e., $\varphi(x) = \varphi(y) \implies x \in \{y, y^{-1}\}$.

Corollary 1 ([HK23]). Let $\psi : \mathbb{B} \mapsto \mathbb{Z}_p$ be a semi-injective function to which we associate the quantity:

$$\varepsilon_\psi = \max_{t \in \mathbb{Z}_p} \Pr_{0 \leq y < 2^\lambda} (\psi(y) = t)$$

Then, the value of ε_ψ is bounded above by $2/p$.

Definition 2 (Bijective Random Oracle). A bijective random oracle, hereby represented by **bRO**, is an idealized public bijection that is accessible, in both directions, via oracles. We model Π as a **bRO** to write the security proof. We write Π and Π^{-1} as oracles. Π is defined as follows:

$$\Pi : \{0, 1\}^\lambda \mapsto [2^\lambda - 1]$$

Note that $f(\pm X_i) = x_i$ due to the symmetry of the elliptic curve with respect to the horizontal axis. As Π corresponds to the bijective random oracle, as per usual, we use two lists, $\mathfrak{L}_\Pi$ and $\mathfrak{L}_{\Pi^{-1}}$ to represent the pair of values $(\alpha, \beta) \in \mathbb{A} \times \mathbb{B}$ such that $\Pi \subseteq \mathbb{A} \times \mathbb{B}$. For a general element of $\mathbb{Z}_p$, we use t which is equals to $\psi(\beta)$ for some $\beta \in \mathbb{B}$. Finally, one can easily write the following relation for $P \in E(\mathbb{F}_q)$ where $\alpha \in \mathbb{A}$ and $\beta \in \mathbb{B}$:

$$\varphi(P) = \alpha, \Pi(\alpha) = \beta \iff \Pi^{-1}(\beta) = \alpha, \psi(\beta) = t \in \mathbb{Z}_p$$

2.3 Identity-Based Signature Schemes

Identity-based signature schemes are a cryptographic primitive derived from the seminal work by Shamir [Sha85]. He first introduced an identity-based setting where the necessity of exchanging private and public keys was removed due to the existence of a *trusted* authority that issues the user's secret key using a master secret. The trusted authority is commonly referred to as the key generation center (KGC) or Trusted Third Party (TTP).

Definition 3 (Identity-based Signature). An identity-based signature scheme IBS is a tuple of four efficient algorithms $\text{IBS} = (\text{Setup}, \text{KeyDeriv}, \text{Sign}, \text{Verify})$ defined as follows:

- i) **Setup**(λ): the **Setup** algorithm is responsible for generating the master key pair (msk , mpk) upon input the security parameter λ
- ii) **KeyDeriv**(msk , id): based on the user's identifier id , the trusted authority performs the key derivation algorithm to obtain the user's secret key, i.e., $\text{sk}_{\text{id}} \leftarrow \text{KeyDeriv}(\text{msk}, \text{id})$, and sends it back to the user
- iii) **Sign**(id , m , sk_{id}): the signing algorithms proceeds as usual producing a signature σ on the digest of m and id with the user's private signing key sk_{id}
- iv) **Verify**(σ , id , m , mpk): the verification algorithm outputs the value $b = 1$ if the signature is valid over the message m and the id using mpk as public verification key, and $b = 0$ otherwise.

The security model used to prove the security of the IBS is a variant of the known existential unforgeability under adaptive chosen-message attack [GM84], which is extended by appending the identity attack [GS02].

Definition 4 (EUF-ID-CMA Game). An adversary $\mathcal{A}$ is said to break the existential unforgeability under chosen-message and identity attack game against the identity-based signature scheme IBS if, after receiving the master public key, he can produce a forged signature on a new identity and a new identity-message pair that has never been queried. The adversary is given access to two oracles, the extraction and signing oracles ($\mathcal{O}_{\text{ext}}$ and $\mathcal{O}_{\text{sig}}$, respectively). The first, upon receiving a query on the id , returns the user's private key sk_{id} , while the second produces a signature after being queried on a pair (m, id) . The advantage of $\mathcal{A}$ winning the game is given by

$$\text{Adv}_{\mathcal{A}, \text{IBS}}^{\text{euf-id-cma}}(\lambda) = \Pr \left[1 \leftarrow \text{Verify}(\sigma^*, \text{id}^*, \text{m}^*) \mid \begin{array}{l} (\sigma^*, \text{id}^*, \text{m}^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{ext}}, \mathcal{O}_{\text{sig}}}(\text{mpk}), \\ \langle \text{id}^* \rangle \notin \mathcal{L}_{\text{ext}}, \langle \text{id}^*, \text{m}^* \rangle \notin \mathcal{L}_{\text{sig}}, \\ (\text{mpk}, \text{msk}) \leftarrow \text{Setup}(\lambda) \end{array} \right] \quad (1)$$

A weaker, but still useful security notion, is the *unforgeability under key-only attacks*. In this setting, the games proceed as a usual chosen-message attack game with one difference: the attacker is not allowed to pose signing queries to the signing oracle. We define this notion in the identity-based setting as follows:

Definition 5 (UF-ID-KOA Game). An adversary $\mathcal{A}$ is said to break the unforgeability under key-only and identity attack game against the identity-based signature scheme IBS if, after receiving the master public key, he can produce a forged signature on a new identity and a new identity-message pair that has never been queried. The adversary is given access to the extraction oracle only, which performs as Definition 4. The advantage of $\mathcal{A}$ winning the game is given by

$$\text{Adv}_{\mathcal{A}, \text{IBS}}^{\text{uf-id-koa}}(\lambda) = \Pr \left[1 \leftarrow \text{Verify}(\sigma^*, \text{id}^*, \text{m}^*) \mid \begin{array}{l} (\sigma^*, \text{id}^*, \text{m}^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{ext}}}(\text{mpk}), \\ \langle \text{id}^* \rangle \notin \mathcal{L}_{\text{ext}}, \\ (\text{mpk}, \text{msk}) \leftarrow \text{Setup}(\lambda) \end{array} \right] \quad (2)$$

2.4 Cryptographic Assumptions

Definition 6 (Discrete Logarithm Problem). Let $(\mathbb{G}, G, p)$ be a prime-order group. An adversary $\mathcal{A}$ is said to $(t_{\text{dl}}, \varepsilon_{\text{dl}})$ -break the discrete logarithm problem if, upon receiving $X := xG$ from challenger $\mathcal{C}$, it runs in time $t \leq t_{\text{dl}}$ and has an advantage

$$\text{Adv}_{\mathcal{A}, \mathcal{C}}^{\text{dlp}}(\lambda) := \Pr \left[\hat{x} = x \mid \begin{array}{l} \hat{x} \leftarrow \mathcal{A}(X), \\ x \leftarrow \mathbb{Z}_p^*, X = xG, \\ (\mathbb{G}, G, p) \leftarrow \text{G}(\lambda) \end{array} \right] = \varepsilon_{\text{dl}} \quad (3)$$

Definition 7 (Semi-Logarithm Problem, [FKP17]). Let $(\mathbb{G}, G, p)$ be a prime-order group, f a function mapping elements from $\mathbb{G}^*$ to $\mathbb{Z}_p$, and $\rho_0, \rho_1 : \mathbb{Z}_p^2 \mapsto \mathbb{Z}_p$ functions. An adversary $\mathcal{A}$ is said to $(t_{\text{slp}}, \varepsilon_{\text{slp}})$ -break the semi-logarithm problem in $\mathbb{G}$ with respect to f, ρ_0, ρ_1 if, upon receiving $X := xG$ from challenger $\mathcal{C}$, it runs in time $t \leq t_{\text{slp}}$ and has an advantage

$$\text{Adv}_{\mathcal{A}, \mathcal{C}}^{\text{slp}}(\lambda) := \Pr \left[v = f(\rho_0(u, v)G + \rho_1(u, v)X) \mid \begin{array}{l} (u, v) \leftarrow \mathcal{A}(X), \\ x \leftarrow \mathbb{Z}_p^*, X = xG, \\ (\mathbb{G}, G, p) \leftarrow \mathcal{G}(\lambda) \end{array} \right] = \varepsilon_{\text{slp}} \quad (4)$$

For both Definitions 6 and 7, we consider that group $\mathbb{G}$ is represented by the elliptic curve $E(\mathbb{F}_q)$ such that its order is equal to p .

Definition 8 (Collision Resistance of a Hash Function). Let $H : \{0, 1\}^* \mapsto \mathbb{Z}_p$ be a hash function. An adversary $\mathcal{A}$ is said to $(t_{\text{cr}, H}, \varepsilon_{\text{cr}, H})$ -break the collision resistance of hash function H if it finds two distinct messages with equal digests in time $t \leq t_{\text{cr}, H}$ and with advantage

$$\text{Adv}_{\mathcal{A}, H}^{\text{cr}}(\lambda) = \Pr \left[H(m_1) = H(m_0) \mid \begin{array}{l} m_0 \neq m_1, \\ m_0, m_1 \in \{0, 1\}^*, \\ (m_0, m_1) \leftarrow \mathcal{A}(\cdot) \end{array} \right] = \varepsilon_{\text{cr}, H} \quad (5)$$

Definition 9 (ψ -Relative Division Resistance of a Hash Function, [FKP16]). Let $H : \{0, 1\}^* \mapsto \mathbb{Z}_p$ be a hash function, $\mathbb{B}$ a set, and $\psi : \mathbb{B} \mapsto \mathbb{Z}_p$ a function. Let $\mathbb{B}^* = \psi^{-1}(\mathbb{Z}_p^*)$. We say an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ $(t_{\text{dr}, H}, \varepsilon_{\text{dr}, H})$ -breaks the ψ -relative division resistance of H if it runs in time $t \leq t_{\text{dr}, H}$ and has an advantage

$$\text{Adv}_{\mathcal{A}, H}^{\text{dr}}(\lambda) = \Pr \left[\frac{H(m_1)}{\psi(\beta_1)} = \frac{H(m_0)}{\psi(\beta_0)} \mid \begin{array}{l} \beta_0, \beta_1 \leftarrow \mathbb{B}^*, \\ (m_0, \Gamma) \leftarrow \mathcal{A}_1(\beta_0), \\ m_1 \leftarrow \mathcal{A}_2(\Gamma, \beta_1) \end{array} \right] = \varepsilon_{\text{dr}, H} \quad (6)$$

Here, Γ is some arbitrary state information passed from $\mathcal{A}_1$ to $\mathcal{A}_2$.

Lemma 1 ([FKP16]). If an attacker can find messages m and m' (not necessarily distinct) such that, for two values, β and β' , the relation $\frac{H(m)}{\psi(\beta)} = \frac{H(m')}{\psi(\beta')}$ is obtained, then he/she can produce a forgery on signature $\sigma^* = (u^*, t^*)$ over message m^* . Hence, the adversary is said to break the ψ -division resistance of hash function H .

Proof. Let $\sigma = (s, t)$, $s = r^{-1}(H(m) + tz)$, $t = f(R) = \psi(\beta)$. We show that the attacker can produce a forgery $\sigma^* = (s^*, t^*)$ over message m^* , as follows

$$sr = H(m) + tz \Rightarrow s \frac{r}{t} = \frac{h}{t} + z = \frac{h^*}{t^*} + z \therefore sr \frac{t^*}{t} = h^* + t^*z \xrightarrow{\times G} s \frac{h^*}{h} R = h^*G + t^*Z$$

Thus, $s^* = sh^*/h$ and $\sigma^* = (sh^*/h, t^*)$. $\square$

In terms of analysis of the Schnorr signature scheme, the General Forking Lemma is a historically used mathematical tool. The proof consists on finding collisions in R by manipulating the random oracle to respond queries with values $R := sG - hZ$, $h := H(R, m)$, and extract the signing key z , thus proving a relation between the advantage of forging a signature and solving the discrete logarithm problem. The idea is shown as follows: if the adversary can produce a forgery by rewinding the random oracle $\sigma^* = (R^*, s^*)$, $R = R^*$, on the forged message m^* , then he can extract the discrete logarithm $z := \log_G^G Z$:

$$R = R^* \Rightarrow sG - hZ = s^*G - h^*Z \therefore z = \frac{s^* - s}{h^* - h} \bmod p \quad (7)$$

ECDSA has a similar reasoning. Now the attacker tries to find collisions between the values of the conversion function: assume that two signatures $\sigma = (u, t)$ and $\sigma^* = (u^*, t^*)$ are obtained, respectively, on messages m and m^* , the idea is to find R, R^* such that $R = \eta R^*$, $|\eta| = 1$. This means obtaining the following relation:

$$tZ = t^*Z \Rightarrow (\eta s - s^*) R^* = (h - h^*) G \therefore z = \frac{s^*h - \eta s h^*}{t^*(\eta s - s^*)} \bmod p \quad (8)$$

Where $h := H(m)$, $h^* := H(m^*)$. By modeling the projection function f as a bijective random oracle, we implicitly allow the usage of Theorem 1 to rewind the oracle until conveniently finding a collision (i.e., $R = \eta R^*$) and extracting the discrete logarithm.

3 Galindo-Garcia IBS

The work by Galindo and Garcia proposed a construction of IBS using two concatenated Schnorr signatures, leveraging the simplicity and the efficiency of such signatures [GG09]. While the technique itself is not new, it has been widely studied when applied to different scenarios. Examples range from identity-based encryption schemes to public key infrastructure-related primitives. The scheme is shown in Definition 10 by considering a group $\mathbb{G}$ of prime order p written multiplicatively and generated by g .

Definition 10 (Galindo-Garcia IBS). The scheme is represented by the tuple of four efficient algorithms $\text{IBS}_{\text{Sch.GG}} := (\text{SetUp}, \text{KeyDeriv}, \text{Sign}, \text{Verify})$ and follows the sequence of steps described below:

- i) **SetUp**(λ): upon receiving the security parameter and having public access to the DLP instance $\Delta = (g, p, \mathbb{G})$, the algorithm first generates $z \leftarrow \mathbb{Z}_p$ and define $Z := g^z$. Set $\text{msk} = z$ and $\text{mpk} = (\mathbb{G}, p, g, Z, H, G)$ where H and G are hash functions mapping from $\{0, 1\}^*$ to $\mathbb{Z}_p$. It finally returns (mpk, msk)
- ii) **KeyDeriv**(msk, id): using the master public key and the user's identifier, the trusted authority samples $r \leftarrow \mathbb{Z}_p$, sets $R := g^r$ and obtains the digest $h := H(R, \text{id})$. Finally, it defines the user's secret key $\text{sk}_{\text{id}} := (y, R)$ where $y := r + zh \bmod p$;
- iii) **Sign**($\text{id}, m, \text{sk}_{\text{id}}$): on inputs $\text{sk}_{\text{id}} = (y, R)$ and id , the user signs the hash of the message m by setting $A := g^a$ where $a \leftarrow \mathbb{Z}_p$, and obtaining the signature $\sigma = (A, b, R) \in \mathbb{G} \times \mathbb{Z}_p \times \mathbb{G}$ where $b := a + yh' \bmod p$, $h' := G(\text{id}, A, m)$;
- iv) **Verify**($\sigma, \text{id}, m, \text{mpk}$): the verification algorithm is done by first calculating both digests, i.e., $h = H(R, \text{id})$ and $h' = G(\text{id}, A, m)$ and then evaluating if the following equality holds:

$$g^b \stackrel{?}{=} A (RZ^h)^{h'}$$

This scheme is proven secure under the EUF-ID-CMA game by constructing two reductions that solve the discrete logarithm problem using the rewinding technique related to the General Forking Lemma (Theorem 1) [BN06] and to the Multiple Forking Lemma (Theorem 2) [BPW12]. It is worth mentioning that the work by Chatterjee et al. highlights some of the problems in the formal analysis and proposes a correct security proof. We refer to the author's original work for a more detailed discussion [CKK13]. All in all, the Galindo-Garcia IBS offers a smaller number of exponentiations and shorter signatures compared to [Her06, BLMQ05], relatively recent work at the time, and represents an attractive option in resource-constrained scenarios.

Theorem 1 (General Forking Lemma). Fix $q \in \mathbb{Z}^+$ and a set $\mathbb{S}$ such that $|\mathbb{S}| \geq 2$. Let $\mathcal{W}$ be a randomized algorithm that on input string x and elements $s_1, \dots, s_q \in \mathbb{S}$ returns a pair (I, σ) consisting of an integer $0 \leq I \leq q$ and a string σ . The forking algorithm $\mathcal{F}_{\mathcal{W}}$ associated to $\mathcal{W}$. The algorithm $\mathcal{F}_{\mathcal{W}}(x)$ returns $(1, \sigma_0, \sigma_1)$ in case of success and $(0, \perp, \perp)$ in case of failure. Let G_1 be a randomized algorithm that takes no input and returns a string. Let

$$\text{acc} := \Pr[x \leftarrow G_1; s_1^0, \dots, s_\gamma^0 \leftarrow \mathbb{S}; (I_0, \sigma_0) \leftarrow \mathcal{W}(x, s_1^0, \dots, s_\gamma^0) | I_0 \geq 1]$$

And

$$\text{gfrk} := \Pr[x \leftarrow G_1; (b, \sigma_0, \sigma_1) \leftarrow \mathcal{F}_{\mathcal{W}}(x) | b = 1]$$

Then

$$\text{gfrk} \geq \text{acc} \left(\frac{\text{acc}}{\gamma} - \frac{1}{|\mathbb{S}|} \right)$$

Theorem 2 (Multiple Forking Lemma). Fix a positive integer q and a set $\mathbb{S}$ with at least two elements. Let n be an odd integer and $\mathcal{M}_{\mathcal{W},n}(x)$ be the multiple forking algorithm over $\mathcal{W}$ that returns the tuple (I, J, σ) with $0 \leq J < I \leq q$, $q \in \mathbb{Z}_{>0}$. On the other hand, $\mathcal{M}$ returns $(1, \{\sigma_0, \dots, \sigma_n\})$ in case of success and $(0, \perp)$. Let G_1 be a randomized algorithm that takes no input and returns a string, and acc is defined as follows:

$$\text{acc} := \Pr[x \leftarrow G_1; s_1^0, \dots, s_\gamma^0 \leftarrow \mathbb{S}; (I_0, \sigma_0) \leftarrow \mathcal{W}(x, s_1^0, \dots, s_\gamma^0) | I_0 \geq 1 \cap J_0 \geq 1]$$

And

$$\text{mfrk} := \Pr[x \leftarrow G_1; (b, \text{results}) \leftarrow \mathcal{M}_{\mathcal{W},n}(x) | b = 1]$$

Then

$$\text{mfrk} \geq \text{acc} \left(\frac{\text{acc}^n}{\gamma^{2n}} - \frac{n}{|\mathbb{S}|} \right)$$

4 Proposed Scheme

Definition 11. Our identity-based signature scheme $\text{IBS}_{\text{ecdsa}} = (\text{Setup}, \text{KeyDeriv}, \text{Sign}, \text{Verify})$ is a tuple of four efficient algorithms defined as follows:

- i) **Setup** (λ): upon input λ and public access to a DLP-instance $\Delta := (E(\mathbb{F}_q), q, G, p)$, it generates the master key pair $(\text{msk}, \text{mpk}) := (z, Z := zG)$ such that $z \leftarrow \mathbb{Z}_p^*$
- ii) **KeyDeriv** (msk, id): the trusted authority generates a random nonce r , sets $R := rG$, calculates $t := f(R)$, $h := H(\text{id})$, and $u := r^{-1}(h + tz) \bmod p$. $\text{sk}_{\text{id}} = (u, t)$ is returned;
- iii) **Sign** ($\text{sk}_{\text{id}}, m, \text{id}$): the user takes u as private signing key, $R := u^{-1}(hG + tZ)$ (correctness of the first signature) as generator, and the verification key $U := uR$. The rest of the algorithm proceeds as usual.
- iv) **Verify** ($\sigma, m, \text{mpk}, \text{id}$): σ is parsed into (u', t', R) to obtain $t = f(R)$, $h := H(\text{id})$, and $h' := G(m, \text{id})$. Using $Z := \text{mpk}$, it recovers $R' = u'^{-1}(h'R + t'hG + t'tZ)$ and outputs the boolean value of $t' \stackrel{?}{=} f(R')$.

Part of the importance of this work lies in some aspects when considering the usage of ECDSA-like schemes and identity-based signature schemes with hierarchical delegation of public keys. Usual ECDSA performs, during verification phase, the following steps considering a signature $\sigma = (s, t)$ on message m : (i) $h := H(m)$, (ii) $R := s^{-1}(hG + tZ)$ with Z being the public verification key, and (iii) outputs the boolean value of $f(R) \stackrel{?}{=} t$.

When considering real-world implementations, for example, information used as input of a hash function generally corresponds to messages, public keys, and commitments to avoid related-key attacks [BK03, MSM⁺16]. However, when embedding the commitment into the hash function, considering, for instance, a construction analog to Schnorr, the verification process becomes impossible since R is recovered based on h , in this case, equals to $H(R, m)$.

A solution to this problem would be to consider using the public key Z of the signer alongside the message to be signed m instead. In such a case, the digest can be calculated so that the R is recovered. The signing algorithm could be written as Figure 3. We denote this first variant of our scheme as $\text{IBS}_{\text{ecdsa}}^{\text{v1}}$. Obtaining an ECDSA signature during the key extraction and signing phase is impossible to achieve simultaneously since verification requires more information to be carried out than what is contained in the signature. In this case, the signing algorithm would proceed as in the right column of Figure 3. However, by applying this reasoning to the signing algorithm, verification would depend on additional information to be carried, and consequently, more complex calculations would be required.

SetUp(λ)	Sign ($\text{sk}_{\text{id}}, m, \text{id}$)
1 : $z \leftarrow \mathbb{Z}_p^*$	1 : $(u, t) := \text{sk}_{\text{id}}$
2 : $\text{msk} := z$	2 : $Z := \text{mpk}$
3 : $Z := zG$	3 : $h := H(\text{id})$
4 : $\text{mpk} := Z$	4 : $R := u^{-1}(hG + tZ)$
5 : return (mpk, msk)	5 : $U := uR$
KeyDeriv (msk, id)	6 : $r' \leftarrow \mathbb{Z}_p$
1 : $z := \text{msk}$	7 : $R' := r'R$
2 : $r \leftarrow \mathbb{Z}_p$	8 : if $R' = \mathcal{O}$: abort
3 : $R := rG$	9 : $t' = f(R')$
4 : if $R = \mathcal{O}$: abort	10 : if $t' = 0$: abort
5 : $t = f(R)$	11 : $h' := G(m, \text{id})$
6 : if $t = 0$: abort	12 : $s' := r'^{-1}(h' + t'u) \bmod n$
7 : $h := H(\text{id})$	13 : return $\sigma = (s', t', R)$
8 : $u := r^{-1}(h + tz) \bmod p$	Verify ($\sigma, m, \text{mpk}, \text{id}$)
9 : return $\text{sk}_{\text{id}} = (u, t)$	1 : $(s', t', R) := \sigma$
	2 : if $t' = 0 \cup s' = 0$: abort
	3 : $Z := \text{mpk}$
	4 : $h := H(\text{id})$
	5 : $h' := G(m, \text{id})$
	6 : $t := f(R)$
	7 : $R' := s'^{-1}(h'R + t'(hG + tZ))$
	8 : if $t' \neq f(R')$:
	9 : return false
	10 : return true

Figure 2: Complete description of $\text{IBS}_{\text{ecdsa}}$.

Also, in terms of security analysis, it would be required to find some other mathematical result instead of the Forking Lemma [PS00] when dealing with H since the fact of using

mpk as an argument would not allow rewinding. Even though one single modification in the key derivation algorithm could be used to obtain an actual ECDSA signature as a private signing key, the implied consequences would still be an object of future study. Another possible implementation would be to use signatures as $\sigma = (u, R)$ with $f(R) = t$, thus, *implicitly*, obtaining a conventional ECDSA signature. By using this format while keeping commitment R as input of the hash function H , we would obtain the verification equation as follows: $(u, R) := \sigma$, $h := H(R, \text{id})$, $t := f(R)$, $uR \stackrel{?}{=} hG + tX$. This version we denote by $\text{IBS}_{\text{ecdsa}}^{v_2}$ which is a Schnorr-like signature as the one proposed by Galindo and Garcia. Using u as private key, one can define $U = uR$, samples $r' \leftarrow \mathbb{Z}_p$, sets $R' = r'R$, obtain $t' = f(R')$ and $h' = G(U, m, \text{id})$, finally setting $u' = r'^{-1}(h' + t'u) \bmod n$. The signature produced would be $\sigma = (u', R', R)$. Verification holds if $u'R' \stackrel{?}{=} h'R + t'U$ for $U \stackrel{?}{=} hG + tX$, $t = f(R)$. Note that the produced signature can still be of the form $\sigma = (u', t', R)$, an even closer variant of ECDSA. In such case, verification would consist of extracting R' by means of $R' = u'^{-1}(h'R + t'U)$ and comparing $t' \stackrel{?}{=} f(R')$. This variant would not work if $h' = G(R', m, \text{id})$ since a circularity would be involved and, thus, only $\sigma = (u', R', R)$ would allow a possible variant.

Finally, a last discussed variant (hereby denoted by $\text{IBS}_{\text{ecdsa}}^{v_3}$) would consist in the following modifications to the proposed scheme: (i) during the **KeyDeriv** algorithm the obtained private key would be of the form $\text{sk}_{\text{id}} := (u, t, R)$ where $t = f(R)$, a ECDSA-like signature over $h := H(R, \text{id})$ and; (ii) the produced signature σ over $h' := G(R', m, \text{id})$ would be of the form (u', R', R) , $R' := r'R$, $r' \leftarrow \mathbb{Z}_p$ thus; (iii) verification would require solely the evaluation of $u'R' \stackrel{?}{=} h'R + t'hG + t'tZ$. This method, although allowing the least amount of operations and, especially, no inversion over $\mathbb{Z}_p$, requires an even more convoluted security proof.

$\text{IBS}_{\text{ecdsa}}^{v_1}.\text{KeyDeriv}(\text{msk}, \text{id})$	$\text{IBS}_{\text{ecdsa}}^{v_1}.\text{Sign}(\text{sk}_{\text{id}}, m, \text{id})$
1 : $z := \text{msk}$	1 : $(u, t, R) := \text{sk}_{\text{id}}$
2 : $Z := zG$	2 : $U := uR$
3 : $r \leftarrow \mathbb{Z}_p$	3 : $r' \leftarrow \mathbb{Z}_p$
4 : $R := rG$	4 : $R' := r'R$
5 : if $R = \mathcal{O}$: abort	5 : if $R' = \mathcal{O}$: abort
6 : $t = f(R)$	6 : $t' = f(R')$
7 : if $t = 0$: abort	7 : if $t' = 0$: abort
8 : $h := H(Z, \text{id})$	8 : $h' := G(U, m, \text{id})$
9 : $u := r^{-1}(h + tz) \bmod n$	9 : $u' := r'^{-1}(h' + t'u) \bmod n$
10 : return $\text{sk}_{\text{id}} = (u, t, R)$	10 : return $\sigma = (u', t', R)$

Figure 3: The modified key derivation algorithm produces the signer's private key as an ECDSA-like signature using the public keys as inputs of the hash functions.

5 Security Proof

In this section, we show the security proof of our scheme $\text{IBS}_{\text{ecdsa}}$ by proving two theorems. The first shows how unforgeability under key-only attack in the identity-based setting implies existential unforgeability under chosen-message and identity attacks, and the second shows how the hardness of the semi-logarithm problem implies unforgeability under key-only and identity attacks.

Theorem 3. *Under the bijective random oracle model, if exists a p.p.t. adversary $\mathcal{F}$ that $(t, \varepsilon, q_{\text{ext}}, q_{\text{sig}}, q_{\Pi})$ -breaks existential unforgeability of $\text{IBS}_{\text{ecdsa}}$, then exists a p.p.t. adversary $\mathcal{F}_0$ that (t', ε') -breaks existential unforgeability of $\text{IBS}_{\text{ecdsa}}$ under key-only and identity attack and adversaries, $\mathcal{A}_1$ and $\mathcal{A}_2$, that $(t_{\text{cr,H}}, \varepsilon_{\text{cr,H}})$ and $(t_{\text{cr,G}}, \varepsilon_{\text{cr,G}})$ -break collision resistance of H and G , respectively, where*

$$\varepsilon \leq \varepsilon' + \varepsilon_{\text{cr,H}} + \varepsilon_{\text{cr,G}} + \frac{(q_{\Pi} + \mathcal{Q})(q_{\text{ext}} + 2q_{\text{sig}})}{(p-1)/2 - \mathcal{Q}} \quad (9)$$

With $t' \geq t + \mathcal{O}(q_{\text{sig}} + q_{\text{ext}})$, $t_{\text{cr}} := t_{\text{cr,H}} + t_{\text{cr,G}} \geq t + \mathcal{O}(q_{\text{ext}} + q_{\text{sig}})$, and $\mathcal{Q} = q_{\text{ext}} + q_{\text{sig}} + q_{\Pi}$, q_{ext} .

Proof. **Game $\mathbf{G}_0^{\mathcal{F}}$.** This is the usual EUF-ID-CMA game, hence: $\Pr[\mathbf{G}_0^{\mathcal{F}} \Rightarrow 1] = \text{Adv}_{\mathcal{F}, \text{IBS}_{\text{ecdsa}}}^{\text{euf-id-cma}}(\lambda) \therefore \Pr[\mathbf{G}_0^{\mathcal{F}} \Rightarrow 1] = \varepsilon$.

Init(λ)	$\text{O}_{\text{sig}}(\text{id}_j, m_j)$	Finalize($\sigma^*, m^*, \text{id}^*, \text{mpk}$)
1 : $\Pi : \mathbb{A} \rightarrow \mathbb{B}$	1 : if $\langle m_j, \text{id}_j \rangle \in \mathcal{L}_{\text{sig}}$:	1 : for $\langle m, \text{id} \rangle \in \mathcal{L}_{\text{sig}}$
2 : $z \leftarrow \$ \mathbb{Z}_n^*$	2 : $\sigma_j := \text{O}_{\text{sig}}(m_j, \text{id}_j)$	2 : if $\text{G}(m^*, \text{id}^*) = \text{G}(m, \text{id})$:
3 : $Z := zG$	3 : return σ_j	3 : abort
4 : $\text{mpk} := Z$	4 : $\text{sk}_{\text{id}_j} := \text{O}_{\text{ext}}(\text{id}_j)$	4 : endfor
5 : return mpk	5 : $(u_j, t_j) := \text{sk}_{\text{id}_j}$	5 : for $\langle \text{id} \rangle \in \mathcal{L}_{\text{ext}}$
bRO (α)	6 : $h_j := \text{H}(\text{id}_j)$	6 : if $\text{H}(\text{id}^*) = \text{H}(\text{id})$:
1 : return $\Pi(\alpha)$	7 : $Z := \text{mpk}$	7 : abort
bRO ⁻¹ (β)	8 : $R_j := u_j^{-1}(h_jG + t_jZ)$	8 : endfor
1 : return $\Pi^{-1}(\beta)$	9 : $U_j := u_jR_j$	9 : if $\text{id}^* \in \mathcal{L}_{\text{ext}}$:
$\text{O}_{\text{ext}}(\text{id}_i)$	10 : $r'_j \leftarrow \$ \mathbb{Z}_n$	10 : abort
1 : if $\langle \text{id}_i \rangle \in \mathcal{L}_{\text{ext}}$:	11 : $R'_j := r'_jR_j$	11 : if $\langle \text{id}^*, m^* \rangle \in \mathcal{L}_{\text{sig}}$:
2 : $\text{sk}_{\text{id}_i} := (u_i, t_i)$	12 : if $R'_j = \mathcal{O}$:	12 : abort
3 : return sk_{id_i}	13 : abort	13 : $Z := \text{mpk}$
4 : $r_i \leftarrow \$ \mathbb{Z}_n$	14 : $\alpha'_j := \varphi(R'_j)$	14 : $(u', t', R) := \sigma^*$
5 : $R_i := r_iG$	15 : $\beta'_j := \Pi(\alpha'_j)$	15 : $\alpha := \varphi(R)$
6 : if $R_i = \mathcal{O}$:	16 : $t'_j := \psi(\beta'_j)$	16 : $\beta := \Pi(\alpha)$
7 : abort	17 : if $t'_j = 0$:	17 : $t := \psi(\beta)$
8 : $\alpha_i \leftarrow \varphi(R_i)$	18 : abort	18 : $h := \text{H}(\text{id}^*)$
9 : $\beta_i \leftarrow \Pi(\alpha_i)$	19 : $h'_j := \text{G}(m_j, \text{id}_j)$	19 : $U := hG + tZ$
10 : $t_i \leftarrow \psi(\beta_i)$	20 : $s'_j := h'_j + t'_ju_j$	20 : if $U = \mathcal{O}$
11 : if $t_i = 0$:	21 : if $s'_j = 0$:	21 : abort
12 : abort	22 : abort	22 : $h' := \text{G}(m^*, \text{id}^*)$
13 : $h_i := \text{H}(\text{id}_i)$	23 : $u'_j := s'_j/r'_j$	23 : $R' \stackrel{?}{=} u'^{-1}(h'R + t'U)$
14 : $s_i := h_i + t_iz$	24 : $\sigma_j := (u'_j, t'_j, R_j)$	24 : $\alpha' := \varphi(R')$
15 : if $s_i = 0$:	25 : return σ_j	25 : $\beta' := \Pi(\alpha')$
16 : abort		26 : return $t' \stackrel{?}{=} \psi(\beta')$
17 : $u_i = s_i/r_i$		
18 : $\text{sk}_{\text{id}_i} = (u_i, t_i)$		
19 : return sk_{id_i}		

Figure 4: Specification of the oracles used for games $\mathbf{G}_0^{\mathcal{F}}$ and $\mathbf{G}_1^{\mathcal{F}}$.

Game $\mathbf{G}_1^{\mathcal{F}}$. Here, we consider the probability of finding a hash collision during the **Finalize** procedure. Neither hash functions are modeled as a random oracle, thus avoiding the usual strategies for modeling repeated (or brand new) queries. The time required is given by $t_{\text{cr}} := t_{\text{cr,H}} + t_{\text{cr,G}} \geq t + \mathcal{O}(q_{\text{ext}} + q_{\text{sig}})$.

$$\Pr[\mathbf{G}_0^{\mathcal{F}} \Rightarrow 1] = \Pr[\mathbf{G}_1^{\mathcal{F}} \Rightarrow 1 \cup \text{collision}^{\text{H}} \cup \text{collision}^{\text{G}}] \leq \Pr[\mathbf{G}_1^{\mathcal{F}} \Rightarrow 1] + \varepsilon_{\text{cr,H}} + \varepsilon_{\text{cr,G}}$$

Figure 4 describes the procedures and oracles implementations for both Games $\mathbf{G}_0^{\mathcal{F}}$ and $\mathbf{G}_1^{\mathcal{F}}$.

Game $\mathbf{G}_2^{\mathcal{F}}$. In this Game, we introduce abort conditions during queries being made to the bijective random oracle. Figure 5 show the description of Game $\mathbf{G}_2^{\mathcal{F}}$. We divide the analysis into three events described as follows:

Event $\mathbf{S}_1$. When $\mathcal{F}$ queries the **bRO** either directly or implicitly (by querying the extraction or signing oracle), $|\Pi^{\text{O}}|$ increases, thus $|\Pi^{\text{S}}|$ decreases since $\Pi := \Pi^{\text{O}} \cup \Pi^{\text{S}}$. Lines 4 of **bRO** ^{± 1} are executed q_{Π} in total. The number of queries to the signing oracle is bounded by q_{sig} , while the number of queries to the extraction oracle is at most q_{ext} . This gives us the probability of occurrence of event $\mathbf{S}_1$ bounded by:

$$\Pr[\mathbf{S}_1] \leq \frac{q_{\Pi}(q_{\text{ext}} + q_{\text{sig}})}{2^{\lambda} - \mathcal{Q}}. \quad (10)$$

With $\mathcal{Q} = q_{\text{ext}} + q_{\Pi} + q_{\text{sig}}$ being the sum of explicit (q^{expl}) and implicit (q^{impl}) queries made to each oracle, i.e.,

$$\mathcal{Q} \leq \underbrace{\left(q_{\text{ext}}^{\text{expl}} + q_{\Pi}^{\text{impl}}\right)}_{\text{extraction phase}} + \underbrace{\left(q_{\text{sig}}^{\text{expl}} + q_{\text{ext}}^{\text{impl}} + q_{\Pi}^{\text{impl}}\right)}_{\text{signing phase}} + q_{\Pi}^{\text{expl}}.$$

Event $\mathbf{S}_2$. For event $\mathbf{S}_2$ we have, in line 12 of procedure **O_{ext}(id)**, a value bounded above by $(q_{\text{ext}} + q_{\Pi})q_{\text{ext}}/2^{\lambda}$, and for line 9, using the semi-injective characteristic of φ , bounded by $2(q_{\text{ext}} + q_{\Pi})q_{\text{ext}}/(p-1)$. Hence, we can show that $2/(p-1) \geq 2^{-\lambda}$ and the total probability of event $\mathbf{S}_2$ is bounded above by

$$\Pr[\mathbf{S}_2] \leq \frac{q_{\text{ext}}(q_{\text{ext}} + q_{\Pi})}{2^{\lambda}} + \frac{q_{\text{ext}}(q_{\text{ext}} + q_{\Pi})}{\frac{p-1}{2}} \leq \frac{2q_{\text{ext}}(q_{\text{ext}} + q_{\Pi})}{\frac{p-1}{2}}. \quad (11)$$

Event $\mathbf{S}_3$. Due to the symmetry between $\mathbf{S}_2$ and $\mathbf{S}_3$, we have:

$$\Pr[\mathbf{S}_3] \leq \frac{q_{\text{sig}}\mathcal{Q}}{\frac{p-1}{2}}. \quad (12)$$

Combining Equation 10, 11, and 12, using $q \leq 2^{\lambda+1} + 1$, and $q_{\text{ext}} + q_{\Pi} \leq q_{\text{sig}}$, we have:

$$\Pr[\mathbf{G}_1^{\mathcal{F}} \Rightarrow 1] \leq \Pr[\mathbf{G}_2^{\mathcal{F}} \Rightarrow 1] + \frac{(q_{\Pi} + \mathcal{Q})(q_{\text{ext}} + 2q_{\text{sig}})}{\frac{p-1}{2} - \mathcal{Q}}.$$

Game $\mathbf{G}_3^{\mathcal{F}}$: from the specifications of Game 3 in Figure 6 and Game 2 in Figure 5 the difference resides at the different moments in which the abort can happen. In Game 2, the α value is defined accordingly to $\varphi(R_i)$ and then β_i is sampled over $\mathbb{B}$. In Game 3, the order of operations is inverted. Since the values of α and β are randomly chosen, both Games 2 and 3 are indistinguishable, allowing us to write:

$$\Pr[\mathbf{G}_3^{\mathcal{F}} \Rightarrow 1] = \Pr[\mathbf{G}_2^{\mathcal{F}} \Rightarrow 1]$$

Init(λ)	O _{ext} (id _i)	O _{sig} (id _j , m _j)
1 : $(\Pi^O, \Pi^S) = (\emptyset, \emptyset)$	1 : if $\langle \text{id}_i \rangle \in \mathcal{L}_{\text{ext}}$:	1 : if $\langle \text{m}_j, \text{id}_j \rangle \in \mathcal{L}_{\text{sig}}$:
2 : $z \leftarrow \mathbb{Z}_p^*$	2 : $\text{sk}_{\text{id}_i} := (u_i, t_i)$	2 : $\sigma_j := \text{O}_{\text{sig}}(\text{m}_j, \text{id}_j)$
3 : $Z := zG$	3 : return sk_{id_i}	3 : return σ_j
4 : return Z	4 : $r_i \leftarrow \mathbb{Z}_p$	4 : $\text{sk}_{\text{id}_j} := \text{O}_{\text{ext}}(\text{id}_j)$
bRO (α)	5 : $R_i := r_i G$	5 : $(u_j, t_j) := \text{sk}_{\text{id}_j}$
1 : if $(\alpha, \cdot) \in \Pi$:	6 : if $R_i = \mathcal{O}$:	6 : $h_j := H(\text{id}_j)$
2 : return $\Pi(\alpha)$	7 : abort	7 : $Z := \text{mpk}$
3 : $\beta \leftarrow \mathbb{B} \setminus \text{Rng}(\Pi^O)$	8 : $\alpha_i \leftarrow \varphi(R_i)$	8 : $R_j := u_j^{-1}(h_j G + tZ)$
4 : if $(\cdot, \beta) \in \Pi^S$:	9 : if $(\alpha_i, \cdot) \in \Pi$:	9 : $U_j := u_j R_j$
5 : abort	10 : abort	10 : $r'_j \leftarrow \mathbb{Z}_p$
6 : $\Pi^O \leftarrow \Pi^O \cup \{(\alpha, \beta)\}$	11 : $\beta_i \leftarrow \mathbb{B}$	11 : $R'_j := r'_j R_j$
7 : return β	12 : if $(\cdot, \beta_i) \in \Pi$:	12 : if $R'_j = \mathcal{O}$:
bRO ⁻¹ (β)	13 : abort	13 : abort
1 : if $(\cdot, \beta) \in \Pi$:	14 : $\Pi^S \leftarrow \Pi^S \cup \{(\alpha_i, \beta_i)\}$	14 : $\alpha'_j := \varphi(R'_j)$
2 : return $\Pi^{-1}(\beta)$	15 : $t_i \leftarrow \psi(\beta_i)$	15 : if $(\alpha'_j, \cdot) \in \Pi$:
3 : $\alpha \leftarrow \mathbb{A} \setminus \text{Dom}(\Pi^O)$	16 : if $t_i = 0$:	16 : abort
4 : if $(\alpha, \cdot) \in \Pi^S$:	17 : abort	17 : $\beta'_j \leftarrow \mathbb{B}$
5 : abort	18 : $h_i := H(\text{id}_i)$	18 : if $(\cdot, \beta'_j) \in \Pi$:
6 : $\Pi^O \leftarrow \Pi^O \cup \{(\alpha, \beta)\}$	19 : $s_i := h_i + t_i z$	19 : abort
7 : return α	20 : if $s_i = 0$:	20 : $\Pi^S \leftarrow \Pi^S \cup \{(\alpha'_j, \beta'_j)\}$
	21 : abort	21 : $t'_j := \psi(\beta'_j)$
	22 : $u_i = s_i / r_i$	22 : if $t'_j = 0$:
	23 : $\text{sk}_{\text{id}_i} = (u_i, t_i)$	23 : abort
	24 : return sk_{id_i}	24 : $h'_j := G(\text{m}_j, \text{id}_j)$
		25 : $s'_j := h'_j + t'_j u_j$
		26 : if $s'_j = 0$:
		27 : abort
		28 : $u'_j := s'_j / r'_j$
		29 : $\sigma_j := (u'_j, t'_j, R_j)$
		30 : return σ_j

Figure 5: Specification of the oracles used during game $\mathbf{G}_2^{\mathcal{F}}$ with changes with respect to $\mathbf{G}_1^{\mathcal{F}}$ highlighted.

Forger $\mathcal{F}_0$. Game 3 relates to the case where $\mathcal{F}_0$ tries to break the key-only unforgeability of our scheme. The argument is analogue to the one in [FKP16]: assume $\mathcal{F}$ produces a forgery $\sigma^* = (u'^*, t'^*, R^*)$, i.e., $u'^* r'^* = h'^* + t'^* u^*$, on $\langle \text{id}^*, \text{m}^* \rangle$, then we have two cases: $(\alpha'^*, \beta'^*) \in \Pi^S$ or $(\alpha'^*, \beta'^*) \in \Pi^O$. In the first case, the pair was defined in lines 22 or 25 of the extraction or signing oracle, respectively. Meaning, $\mathcal{F}_0$ knows a signature $\sigma_i = (u'_i, t'_i, R_i)$ such that the conditions of Equation 8 occur, and $\mathcal{F}_0$ extracts the user's secret key, thus winning the game. The case when the forged pair $(\alpha, \beta) \in \Pi^O$ follows with little adaptation, since Π^O contains the list of elements updated by querying the bijective random oracle. This means, $\mathcal{F}$ succeeded and so did $\mathcal{F}_0$, i.e., $\Pr[\mathbf{G}_3^{\mathcal{F}} \Rightarrow 1] = \Pr[\mathbf{G}_0^{\mathcal{F}_0} \Rightarrow 1] = \text{Adv}_{\mathcal{F}_0, \text{IBS}_{\text{ecdsa}}}^{\text{uf-id-koa}}(\lambda)$. By combining all previous results, we obtain the desired

result. Note that, for EUF-CMA, taking $q_{\text{ext}} = 0$ and $q_{\Pi} \leq Q/2$, $Q \rightarrow q_{\Pi} + q_{\text{sig}}$ we obtain the result obtained by [FKP16].

$O_{\text{ext}}(\text{id}_i)$	$O_{\text{sig}}(\text{m}_j, \text{id}_j)$
1 : if $\text{id}_i \in \mathfrak{L}_{\text{ext}}$:	1 : if $(\text{id}_j, \text{m}_j) \in \mathfrak{L}_{\text{sig}}$:
2 : $\text{sk}_{\text{id}_i} := (u_i, t_i)$	2 : $\sigma_j := O_{\text{sig}}(\text{m}_j, \text{id}_j)$
3 : return sk_{id_i}	3 : return σ_j
4 : $r_i \leftarrow \$ \mathbb{Z}_n$	4 : $\text{sk}_{\text{id}_j} := O_{\text{ext}}(\text{id}_j)$
5 : $R_i := r_i G$	5 : $(u_j, t_j) := \text{sk}_{\text{id}_j}$
6 : if $R_i = \mathcal{O}$:	6 : $U_j := u_j R_j$
7 : abort	7 : $\beta'_j \leftarrow \$ \mathbb{B}$
8 : $\beta_i \leftarrow \$ \mathbb{B}$	8 : if $(\cdot, \beta'_j) \in \Pi$:
9 : if $(\cdot, \beta_i) \in \Pi$:	9 : abort
10 : abort	10 : $t'_j := \psi(\beta'_j)$
11 : $t_i \leftarrow \psi(\beta_i)$	11 : if $t'_j = 0$:
12 : if $t_i = 0$:	12 : abort
13 : abort	13 : $r'_j \leftarrow \$ \mathbb{Z}_n$
14 : $h_i := H(\text{id}_i)$	14 : $R'_j := r'_j R_j$
15 : $s_i := h_i + t_i z$	15 : if $R'_j = \mathcal{O}$:
16 : if $s_i = 0$:	16 : abort
17 : abort	17 : $h'_j := G(\text{m}_j, \text{id}_j)$
18 : $u_i = s_i / r_i$	18 : $s'_j := h'_j + t'_j u_j$
19 : $\alpha_i \leftarrow \varphi(R_i)$	19 : if $s'_j = 0$:
20 : if $(\alpha_i, \cdot) \in \Pi$:	20 : abort
21 : abort	21 : $u'_j := s'_j / r'_j$
22 : $\Pi^S \leftarrow \Pi^S \cup \{(\alpha_i, \beta_i)\}$	22 : $\alpha'_j := \varphi(R'_j)$
23 : $\text{sk}_{\text{id}_i} = (u_i, t_i)$	23 : if $(\alpha'_j, \cdot) \in \Pi$:
24 : return sk_{id_i}	24 : abort
	25 : $\Pi^S \leftarrow \Pi^S \cup \{(\alpha'_j, \beta'_j)\}$
	26 : $\sigma_j := (u'_j, t'_j, R_j)$
	27 : return σ_j

Figure 6: Specification of $G_3^{\mathcal{F}}$ resulted from **changes** with respect to game $G_2^{\mathcal{F}}$.

□

Theorem 4. *In the bijective random oracle model, if exists an p.p.t adversary $\mathcal{F}$ that $(t, \varepsilon, q_{\text{ext}}, q_{\Pi})$ -breaks the unforgeability under identity and key-only attack of $\text{IBS}_{\text{ecdsa}}$, then exists a p.p.t. adversaries $\mathcal{F}_0$ that (t', ε') -breaks the unforgeability under key-only attack of ECDSA and $\mathcal{B}$ that $(t_{\text{cr}, H}, \varepsilon_{\text{cr}, H})$ -breaks the collision resistance of hash function H where*

$$\varepsilon \leq \left(\sqrt{2q_{\Pi}} + 1 \right) \varepsilon' + \varepsilon_{\text{cr}, G}, \quad t' \geq t + \mathcal{O}(\sqrt{q_{\Pi}})$$

Proof. The game starts with $\mathcal{C}$ generating a master key pair $(\text{msk}, \text{mpk}) \leftarrow \$ \text{IBS}_{\text{ecdsa}}.\text{Init}(\lambda)$ accordingly to $\text{IBS}_{\text{ecdsa}}$'s description (Definition 11), and sending challenge mpk to forger

$\mathcal{F}_0$. The forger is allowed only to pose queries to the bRO. $\mathcal{F}_0$ then passes the **mpk** to forger $\mathcal{F}$. $\mathcal{F}$ is allowed to query the extraction oracle $\mathcal{O}_{\text{ext}}$ on id and the bRO, and attempts to produce a valid forged signature on a brand new message-identity pair accordingly to Definition 5.

The idea of the proof is described as follows. Every query posed by $\mathcal{F}$ to the extraction oracle is seen by $\mathcal{F}_0$, which in turn monitors the values queried in his/her table. We denote this table by $\mathfrak{L}_{\text{ext}, \mathcal{F}_0}$. The same happens when oracle $\text{bRO}^{\pm 1}$ is queried. $\mathcal{F}_0$ keeps track of queries with tables $\mathfrak{L}_{\Pi, \mathcal{F}_0}$ and $\mathfrak{L}_{\Pi^{-1}, \mathcal{F}_0}$. For each query that $\mathcal{F}$ poses to the extraction oracle, the oracle returns $\text{sk}_{\text{id}} := \mathcal{O}_{\text{ext}}(\text{id})$ as specified in Figure 4. $\mathcal{F}_0$ passes the challenge **mpk** to forger $\mathcal{F}$ as the second tries to break unforgeability under identity and key-only attack, i.e., produce a forged signature $\sigma^* = (u'^*, t'^*, R^*)$ on a brand-new identity-message pair.

$\mathcal{F}_0$ wins when *i*) $\mathcal{F}$ outputs a forgery $\sigma_0^* = (u'_0, t'_0, R^*)$, and *ii*) rewinds $\mathcal{F}$ until it obtains another signature $\sigma_1^* = (u'_1, t'_1, R^*)$ on the same u_{id} such that $t'_0 \neq t'_1$, $\text{id}^* \notin \mathfrak{L}_{\text{ext}}$. Hence, using Equation 8 with $r'_1 = \eta r'_0$, $\mathcal{F}_0$ calculates r'_0 and extracts u_{id^*} :

$$r_{0'} = \frac{h'_0 - h'_1}{t'_0(u'_0 - \eta u'_1)} \therefore u_{\text{id}^*} = \frac{u'_0 h'_1 - \eta u'_1 h'_0}{t'_0(\eta u'_1 - u'_0)}. \quad (13)$$

Finally, $\mathcal{F}_0$ uses R^* to calculate $t^* = f(R^*)$, and outputs (u_{id}, t^*) , winning the UF-KOA game. All in all, we can bound the probability of $\mathcal{F}_0$ being successful using the General Forking Lemma:

$$\varepsilon' \geq \text{acc} \left(\frac{\text{acc}}{q_{\Pi}} - \frac{1}{2^{\lambda}} \right)$$

Where $\text{acc} = \varepsilon - \varepsilon' - \varepsilon_{\text{cr}, \text{G}}$. Manipulating the equation, we can consider two cases:

Case 1. If $\frac{\text{acc}}{2q_{\Pi}} < \frac{1}{2^{\lambda}} \leq \frac{\text{acc}}{q_{\Pi}}$, then

$$\varepsilon \leq \varepsilon' + \varepsilon_{\text{cr}, \text{G}} + \frac{2q_{\Pi}}{2^{\lambda}} \quad (14)$$

Case 2. If $\frac{1}{2^{\lambda}} \leq \frac{\text{acc}}{2q_{\Pi}}$, then

$$\varepsilon' \geq \frac{\text{acc}^2}{2q_{\Pi}} \therefore \varepsilon \leq \left(\sqrt{2q_{\Pi}} + 1 \right) \varepsilon' + \varepsilon_{\text{cr}, \text{G}} \quad (15)$$

□

6 Results

In this section, we present the results obtained from algorithm analysis and by simulating identity-based ECDSA and Schnorr signature algorithms with hierarchical public key delegation. We start by analyzing the algorithms (Tables 1 and 2), evaluating the number of operations performed by each scheme.

Algorithmic Analysis. As presented in Section 4, we propose different ECDSA-like identity-based schemes that support signature concatenation employing the Hierarchical Public Key Delegation technique. In Table 1, these different schemes are compared in terms of algorithmic efficiency when considering the following operations: *i*) sum, multiplication, and inversion over $\mathbb{Z}_p$ are represented by **sum**, **mul**, and **inv**, respectively, *ii*) for point addition and multiplication by scalar we use **pad** and **mbs**, and *iii*) for hash operations, we use **hsh**. As expected, our scheme has a higher algorithmic complexity since it lacks the linear property of Galindo-Garcia's scheme.

In terms of signature size $|\sigma|$, Table 2 shows that the obtained ECDSA-like signatures are smaller than Galindo-Garcia IBS, $2|\mathbb{Z}_p^*| + |\mathbb{G}^*|$ instead of $|\mathbb{Z}_p^*| + 2|\mathbb{G}^*|$. This is a trade-off between $\text{IBS}_{\text{ecdsa}}$'s higher algorithmic complexity and smaller signatures due to ECDSA's non-linear structure and more convoluted verification steps regarding the recovery of commitment R to further compare it to $f(R)$ during verification.

Table 1: Algorithmic analysis of key-derivation, sign, and verify algorithms comparing the variants of identity-based ECDSA and [GG09] scheme.

IBS	KeyDeriv	Sign	Verify
IBS_{GG}	mbs + hsh + sum + mul	mbs + hsh + sum + mul	2pad + 5mbs + 2hsh
$\text{IBS}_{\text{ecdsa}}$	mbs + hsh + sum + inv + 2mul	3mbs + 2hsh + 2sum + 2inv + 4mul	5pad + 2mbs + 2hsh + inv
$\text{IBS}_{\text{ecdsa}}^{v1}$	-	3mbs + 2hsh + 2sum + 2inv + 4mul	5pad + 2mbs + 2hsh + inv
$\text{IBS}_{\text{ecdsa}}^{v2}$	-	2mbs + hsh + sum + inv + mul	3pad + 4mbs + 2hsh
$\text{IBS}_{\text{ecdsa}}^{v3}$	-	2mbs + hsh + sum + inv + mul	3pad + 4mbs + 2hsh

Table 2: Comparison of signature's format and size between Galindo-Garcia's IBS and $\text{IBS}_{\text{ecdsa}}$.

	σ	$ \sigma $
IBS_{GG}	(A, b, R)	$ \mathbb{Z}_p + 2 \mathbb{G}^* $
$\text{IBS}_{\text{ecdsa}}$	(u', t', R)	$2 \mathbb{Z}_p + \mathbb{G}^* $
$\text{IBS}_{\text{ecdsa}}^{v1}$	(u', t', R)	$2 \mathbb{Z}_p + \mathbb{G}^* $
$\text{IBS}_{\text{ecdsa}}^{v2}$	(u', t', R)	$2 \mathbb{Z}_p + \mathbb{G}^* $
$\text{IBS}_{\text{ecdsa}}^{v3}$	(u', R', R)	$ \mathbb{Z}_p + 2 \mathbb{G}^* $

Baseline Results. During the experiments, we use a highly optimized elliptic curve library implemented in C to compare our implementations using OpenSSL. Table 3 aims to show these minor differences between ECDSA and Schnorr when considering a native implementation (OpenSSL implements ECDSA in its libraries, but does not implement Schnorr) versus a non-native implementation (Schnorr's signature algorithm with OpenSSL). Our results represent the same tendency when comparing the algorithms in ECDSA and Schnorr. While Schnorr is faster when signing, ECDSA's verification performs faster. Further, based on these codes, we compare the performance of Galindo-Garcia's IBS and ours using OpenSSL. Our scheme performs slightly faster during key derivation and signing algorithms, as we can see in Table 4.

Table 3: Comparison between ECDSA and Schnorr algorithm instantiated over secp256k1 using OpenSSL (our implementation) and the baseline result using the bitcoin-core repository [Bc25] – a highly optimized implementation of both algorithms over elliptic curve secp256k1.

	ECDSA avg. performance in ms		Schnorr avg. performance in ms	
	OpenSSL	Baseline Result	OpenSSL	Baseline Result
KeyGen	2.40 ± 0.38	0.29	2.80 ± 0.52	0.29
Sign	0.89 ± 0.18	0.39	0.61 ± 0.17	0.27
Verify	0.55 ± 0.12	0.58	1.20 ± 0.32	0.62

Table 4: Average performance comparison between the hierarchical delegation of public key technique applied to ECDSA and the Schnorr algorithm.

	IBS _{ecdsa}	IBS _{GG}
KeyDeriv	0.82 ± 0.30	1.23 ± 0.35
Sign	0.66 ± 0.24	0.69 ± 0.18

7 Conclusion

In this paper, we present the first ECDSA-like identity-based signature scheme that allows signature concatenation, employing the Hierarchical Delegation of Public Key technique, which leverages the cyclic property of the group. Our security analysis combines the *existential unforgeability under chosen-message and chosen identity attack game* – EUF-ID-CMA game – by embedding the modeling of the conversion function as a bijective random oracle. We demonstrate that the security of our scheme can be reduced to ECDSA’s unforgeability under key-only attacks in Section 5 (a result by [FKP16]). Additionally, we present an optional security proof in Appendix A, which reduces the security of our scheme to the semi-logarithm problem. We also show a reduction from our scheme to the discrete logarithm problem in two steps, following the strategy of Fersch et al.

In terms of performance, even with a more convoluted algorithmic structure during signing and verification, our scheme works slightly more efficiently than the Galindo-Garcia Schnorr-like identity-based scheme. All in all, it represents an interesting application in cloud-oriented authentication systems due to its support for ECDSA, and opens up interesting possibilities in hierarchical delegation token scenarios and secure credential management.

Acknowledgments

We thank Thales Paiva for important suggestions and discussions during the development of this manuscript. This study was financed in part by Hewlett Packard Enterprise (HPE), and by the following Brazilian research engines: FAPESP (grant 2020/09850-0), CNPq (grant 307732/2023-1), and Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – (Finance Code 001).

References

- [AGM18] Shashank Agrawal, Chaya Ganesh, and Payman Mohassel. Non-interactive zero-knowledge proofs for composite statements. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 643–673. Springer, Cham, August 2018. doi:10.1007/978-3-319-96878-0_22.
- [ANT⁺20] Diego F. Aranha, Felipe Rodrigues Novaes, Akira Takahashi, Mehdi Tibouchi, and Yuval Yarom. Ladderleak: Breaking ecdsa with less than one bit of nonce leakage. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, CCS ’20*, page 225–242, New York, NY, USA, 2020. Association for Computing Machinery. doi:10.1145/3372297.3417268.
- [Bc25] Bitcoin-core. secp256k1: Optimized c library for ec operations on curve secp256k1. <https://github.com/bitcoin-core/secp256k1>, 2025.

- [BCCT12] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference, ITCS '12*, page 326–349, New York, NY, USA, 2012. Association for Computing Machinery. doi:10.1145/2090236.2090263.
- [BH19] Joachim Breitner and Nadia Heninger. Biased nonce sense: Lattice attacks against weak ecDSA signatures in cryptocurrencies. In Ian Goldberg and Tyler Moore, editors, *Financial Cryptography and Data Security*, pages 3–20, Cham, 2019. Springer International Publishing.
- [BK03] Mihir Bellare and Tadayoshi Kohno. A theoretical treatment of related-key attacks: Rka-prps, rka-prfs, and applications. In Eli Biham, editor, *Advances in Cryptology — EUROCRYPT 2003*, pages 491–506, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.
- [BLMQ05] Paulo S. L. M. Barreto, Benoît Libert, Noel McCullagh, and Jean-Jacques Quisquater. Efficient and provably-secure identity-based signatures and sign-encryption from bilinear maps. In Bimal Roy, editor, *Advances in Cryptology - ASIACRYPT 2005*, pages 515–532, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [BN06] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 390–399. ACM Press, October / November 2006. doi:10.1145/1180405.1180453.
- [BPVY00] Ernest Brickell, David Pointcheval, Serge Vaudenay, and Moti Yung. Design validations for discrete logarithm based signature schemes. In Hideki Imai and Yuliang Zheng, editors, *Public Key Cryptography*, pages 276–292, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.
- [BPW12] Alexandra Boldyreva, Adriana Palacio, and Bogdan Warinschi. Secure proxy signature schemes for delegation of signing rights. *Journal of Cryptology*, 25(1):57–115, 01 2012. doi:10.1007/s00145-010-9082-x.
- [BR04] Mihir Bellare and Phillip Rogaway. Code-based game-playing proofs and the security of triple encryption. Cryptology ePrint Archive, Paper 2004/331, 2004. URL: <https://eprint.iacr.org/2004/331>.
- [BR06] Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In Serge Vaudenay, editor, *Advances in Cryptology - EUROCRYPT 2006*, pages 409–426, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
- [Bro02] Daniel R. L. Brown. Generic groups, collision resistance, and ECDSA. Cryptology ePrint Archive, Paper 2002/026, 2002. URL: <https://eprint.iacr.org/2002/026>.
- [Bro05] Daniel R. Brown. Generic groups, collision resistance, and ecDSA. *Des. Codes Cryptography*, 35(1):119–152, 04 2005. doi:10.1007/s10623-003-6154-z.
- [BSCR⁺19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for r1cs. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019*, pages 103–128, Cham, 2019. Springer International Publishing.

- [BZ23] Paulo Barreto and Gustavo Zanon. Blind signatures from zero-knowledge arguments. Cryptology ePrint Archive, Paper 2023/067, 2023. URL: <https://eprint.iacr.org/2023/067>.
- [CFOS25] Gavin Cho, Georg Fuchsbauer, Adam O’Neill, and Marek Sefranek. Schnorr signatures are tightly secure in the rom under a non-interactive assumption. In *Advances in Cryptology – CRYPTO 2025*, Cham, 2025. Springer Nature Switzerland.
- [CKK13] Sanjit Chatterjee, Chethan Kamath, and Vikas Kumar. Galindo-garcia identity-based signature revisited. In Taekyoung Kwon, Mun-Kyu Lee, and Daesung Kwon, editors, *Information Security and Cryptology – ICISC 2012*, pages 456–471, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [CKM23] Elizabeth Crites, Chelsea Komlo, and Mary Maller. Fully adaptive schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 678–709, Cham, 2023. Springer Nature Switzerland.
- [CLR24] Sofia Celi, Shai Levin, and Joe Rowell. Cdl: Proving knowledge of committed discrete logarithms with soundness. In Serge Vaudenay and Christophe Petit, editors, *Progress in Cryptology – AFRICACRYPT 2024*, pages 69–93, Cham, 2024. Springer Nature Switzerland.
- [Eth24] Ethereum. Ethereum Accounts, September 2024. URL: <https://ethereum.org/en/developers/docs/accounts/>.
- [FHLM22] Armando Faz-Hernández, Watson Ladd, and Deepak Maram. Zkattest: Ring and group signatures for existing ecDSA keys. In Riham AlTawy and Andreas Hülsing, editors, *Selected Areas in Cryptography*, pages 68–83, Cham, 2022. Springer International Publishing.
- [FKP16] Manuel Fersch, Eike Kiltz, and Bertram Poettering. On the provable security of (EC)DSA signatures. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1651–1662. ACM Press, October 2016. doi:10.1145/2976749.2978413.
- [FKP17] Manuel Fersch, Eike Kiltz, and Bertram Poettering. On the one-per-message unforgeability of (ec)dsa and its variants. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography*, pages 519–534, Cham, 2017. Springer International Publishing.
- [GG09] David Galindo and Flavio D. Garcia. A schnorr-like lightweight identity-based signature scheme. In Bart Preneel, editor, *Progress in Cryptology – AFRICACRYPT 2009*, pages 135–148, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.
- [Gir91] Marc Girault. Self-certified public keys. In Donald W. Davies, editor, *Advances in Cryptology – EUROCRYPT ’91*, pages 490–497, Berlin, Heidelberg, 1991. Springer Berlin Heidelberg.
- [GKL⁺20] Robert Granger, Thorsten Kleinjung, Arjen K. Lenstra, Benjamin Wesolowski, and Jens Zumbärgel. Computation of a 30750-bit binary field discrete logarithm, 2020. URL: <https://arxiv.org/abs/2008.02717>, arXiv:2008.02717.

- [GM84] Shafi Goldwasser and Silvio Micali. Probabilistic encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984. URL: <https://www.sciencedirect.com/science/article/pii/002200084900709>, doi: [10.1016/0022-0000\(84\)90070-9](https://doi.org/10.1016/0022-0000(84)90070-9).
- [GS02] Craig Gentry and Alice Silverberg. Hierarchical id-based cryptography. In Yuliang Zheng, editor, *Advances in Cryptology — ASIACRYPT 2002*, pages 548–566, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.
- [GS22] Jens Groth and Victor Shoup. On the security of ecdsa with additive key derivation and presignatures. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022*, pages 365–396, Cham, 2022. Springer International Publishing.
- [GWHH25] Yiming Gao, Jinghui Wang, Honggang Hu, and Binang He. Attacking ecdsa with nonce leakage by lattice sieving: Bridging the gap with fourier analysis-based attacks. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 3–34, Singapore, 2025. Springer Nature Singapore.
- [Her06] Javier Herranz. Deterministic identity-based signatures for partial aggregation. *The Computer Journal*, 49(3):322–330, 2006. doi: [10.1093/comjnl/bxh153](https://doi.org/10.1093/comjnl/bxh153).
- [HK23] Dominik Hartmann and Eike Kiltz. Limits in the provable security of ecdsa signatures. In Guy Rothblum and Hoeteck Wee, editors, *Theory of Cryptography*, pages 279–309, Cham, 2023. Springer Nature Switzerland.
- [Lin21] Yehuda Lindell. Fast secure two-party ecdsa signing. *Journal of Cryptology*, 34(4):44, 9 2021. doi: [10.1007/s00145-021-09409-9](https://doi.org/10.1007/s00145-021-09409-9).
- [LV01] Arjen K. Lenstra and Eric R. Verheul. Selecting cryptographic key sizes. *J. Cryptol.*, 14(4):255–293, January 2001. doi: [10.1007/s00145-001-0009-4](https://doi.org/10.1007/s00145-001-0009-4).
- [MPSW19] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. Simple schnorr multi-signatures with applications to bitcoin. *Designs, Codes and Cryptography*, 87(9):2139–2164, Sep 2019. doi: [10.1007/s10623-019-00608-x](https://doi.org/10.1007/s10623-019-00608-x).
- [MSM⁺16] Hiraku Morita, Jacob C. N. Schuldt, Takahiro Matsuda, Goichiro Hanaoka, and Tetsu Iwata. On the security of the schnorr signature scheme and dsa against related-key attacks. In Soonhak Kwon and Aaram Yun, editors, *Information Security and Cryptology - ICISC 2015*, pages 20–35, Cham, 2016. Springer International Publishing.
- [Nak08] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *bitcoin.org*, 2008. URL: <http://bitcoin.org/bitcoin.pdf>.
- [Nat23] National Institute of Standards and Technology. Digital Signature Standard (DSS). Technical Report 186-5, National Institute of Standards and Technology, Feb 2023. Approved February 2023. URL: <https://nvlpubs.nist.gov/nisttpubs/FIPS/NIST.FIPS.186-5.pdf>.
- [Nec94] V. I. Nechaev. Complexity of a determinate algorithm for the discrete logarithm. *Mathematical Notes*, 55(2):165–172, 02 1994. doi: [10.1007/BF02113297](https://doi.org/10.1007/BF02113297).

- [PS00] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *Journal of Cryptology*, 13:361–396, 2000. doi:10.1007/S001450010003.
- [QCY21] Xianrui Qin, Cailing Cai, and Tsz Hon Yuen. One-more unforgeability of blind ecDSA. In Elisa Bertino, Haya Shulman, and Michael Waidner, editors, *Computer Security – ESORICS 2021*, pages 313–331, Cham, 2021. Springer International Publishing.
- [Res18] Eric Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, August 2018. URL: <https://www.rfc-editor.org/info/rfc8446>, doi:10.17487/RFC8446.
- [RSA78] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120–126, February 1978. doi:10.1145/359340.359342.
- [Sch89] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In *Proceedings of the 9th Annual International Cryptology Conference on Advances in Cryptology*, CRYPTO ’89, page 239–252, Berlin, Heidelberg, 1989. Springer-Verlag.
- [Sha85] Adi Shamir. Identity-based cryptosystems and signature schemes. In George Robert Blakley and David Chaum, editors, *Advances in Cryptology*, pages 47–53, Berlin, Heidelberg, 1985. Springer Berlin Heidelberg.
- [Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *Advances in Cryptology — EUROCRYPT ’97*, pages 256–266, Berlin, Heidelberg, 1997. Springer Berlin Heidelberg.
- [SPMLS02] Jacques Stern, David Pointcheval, John Malone-Lee, and Nigel P. Smart. Flaws in applying proof methodologies to signature schemes. In Moti Yung, editor, *Advances in Cryptology — CRYPTO 2002*, pages 93–110, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.
- [WOS⁺25] Anna P. Y. Woo, Alex Ozdemir, Chad Sharp, Thomas Pornin, and Paul Grubbs. Efficient Proofs of Possession for Legacy Signatures . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3291–3308, Los Alamitos, CA, USA, May 2025. IEEE Computer Society. URL: <https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00080>, doi:10.1109/SP61157.2025.00080.
- [XDWL23] Luyao Xu, Zhengyi Dai, Baofeng Wu, and Dongdai Lin. Improved attacks on (ec)dsa with nonce leakage by lattice sieving with predicate. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023, Issue 2:568–586, 2023. URL: <https://tches.iacr.org/index.php/TCHES/article/view/10294>, doi:10.46586/tches.v2023.i2.568-586.

A From UF-ID-KOA to SLP

Theorem 5. *Under the bijective random oracle model, if there exists a forger $\mathcal{F}_0$ that (ε, t, q_Π) -breaks unforgeability of $\text{IBS}_{\text{ecdsa}}$ under chosen identity and key-only attack, then there exists an algorithm $\mathcal{A}$ that $(\varepsilon_{\text{slp}}, t_{\text{slp}})$ -breaks the semi-logarithm problem, where*

$$\varepsilon \leq \varepsilon_{\text{slp}} + \frac{q_\Pi^2}{2^\lambda} + \sqrt{2q_\Pi \varepsilon_{\text{slp}}}$$

With $t_{\text{slp}} \geq t + \mathcal{O}(q_{\Pi})$.

Proof. **Game $\mathbf{G}_0^{\mathcal{F}_0}$.** The first game corresponds to the chosen identity and key-only attack by forger $\mathcal{F}_0$ when interacting with a challenger $\mathcal{C}$. Upon receiving the challenge value Z corresponding to the *master public key*, $\mathcal{F}_0$ tries to output a valid forgery. Finally, one can write:

$$\Pr [\mathbf{G}_0^{\mathcal{F}_0} \Rightarrow 1] = \text{Adv}_{\mathcal{F}_0, \text{IBS}_{\text{ecdsa}}}^{\text{uf-id-koa}}(\lambda) \therefore \Pr [\mathbf{G}_0^{\mathcal{F}_0} \Rightarrow 1] = \varepsilon$$

Game $\mathbf{G}_1$. In this game, we introduce the abort condition during queries to the bijective random oracle and to the extraction oracle.

Init(λ)	$\text{O}_{\text{ext}}(\text{id})$
1 : $(\Pi_{\rightarrow}, \Pi_{\leftarrow}) = (\emptyset, \emptyset)$	(Exactly the same as Figure 5 with
2 : $z \leftarrow \$ \mathbb{Z}_p^*$	$\Pi_O = \Pi_{\rightarrow}$ and $\Pi_S = \Pi_{\leftarrow}$)
3 : $\text{msk} := z$	
4 : $Z := zG$	Finalize($\text{m}^*, \sigma^*, \text{id}^*, \text{mpk}$)
5 : $\text{mpk} := Z$	$Z := \text{mpk}$
6 : return mpk	$(u'^*, t'^*, R^*) := \sigma^*$
$\text{bRO}(\alpha)$	if $u'^* = 0 \cup t'^* = 0 \cup R^* = \emptyset$:
1 : if $(\alpha, \cdot) \in \Pi$:	abort
2 : return $\Pi(\alpha)$	$\alpha^* := \varphi(R^*)$
3 : $\beta \leftarrow \$ \mathbb{B}$	$\beta^* := \psi(\alpha^*)$
4 : if $(\cdot, \beta) \in \Pi$:	$t^* := \psi(\beta^*)$
5 : abort	if $t^* = 0$:
6 : $\Pi_{\rightarrow} \leftarrow \Pi_{\rightarrow} \cup \{(\alpha, \beta)\}$	abort
7 : return β	$h^* := H(\text{id}^*)$
$\text{bRO}^{-1}(\beta)$	$h'^* := G(\text{m}^*, \text{id}^*)$
1 : if $(\cdot, \beta) \in \Pi$:	$R'^* := (u'^*)^{-1}(h'^* R^* + t'^*(h^* G + t^* Z))$
2 : return $\Pi^{-1}(\beta)$	if $R'^* = \emptyset$:
3 : $\alpha \leftarrow \$ \mathbb{A}$	abort
4 : if $(\alpha, \cdot) \in \Pi$:	$\alpha'^* \leftarrow \varphi(R'^*)$
5 : abort	$\beta'^* \leftarrow \Pi(\alpha'^*)$
6 : $\Pi_{\leftarrow} \leftarrow \Pi_{\leftarrow} \cup \{(\alpha, \beta)\}$	if $\psi(\beta'^*) \neq t'^*$
7 : return α	abort
	return true

Figure 7: Games $\mathbf{G}_0^{\mathcal{F}_0}$ and $\mathbf{G}_1^{\mathcal{F}_0}$ during the key-only experiment. Lines highlighted by gray colored boxes represents changes or additions of game $\mathbf{G}_1^{\mathcal{F}_0}$ when compared to game $\mathbf{G}_0^{\mathcal{F}_0}$.

Now, Π is implemented using lazy sampling [BR04] defining two lists, $\Pi_{\rightarrow}$ and $\Pi_{\leftarrow}$, which store, respectively, α and β values. Analogously to Theorem 3, $\Pi := \Pi_{\rightarrow} \cup \Pi_{\leftarrow}$ with abort conditions happening when the two lists contradict each other. However, instead of arguing that Π is composed by lists storing values obtained by means (or not) of programming the oracle, now, the notion of direct queries (*left arrow* subscript) and inverse queries (*right arrow* subscript) is done to allow the use of the General Forking Lemma (Theorem 1). We evaluate the abort probability in game $\mathbf{G}_1^{\mathcal{F}_0}$ accordingly to the

abort condition in lines 5 in both oracles bRO^η , $\eta \in \{\pm 1\}$. Hence:

$$\begin{aligned} \Pr \left[\mathbf{G}_0^{\mathcal{F}_0} \Rightarrow 1 \right] &\leq \Pr \left[\mathbf{G}_1^{\mathcal{F}_0} \Rightarrow 1 \right] + \Pr [\text{abort} \Rightarrow 1] \Rightarrow \\ &\leq \Pr \left[\mathbf{G}_1^{\mathcal{F}_0} \Rightarrow 1 \right] + \frac{q_\Pi^2}{|\mathbb{B}|} \Rightarrow \\ &\leq \Pr \left[\mathbf{G}_1^{\mathcal{F}_0} \Rightarrow 1 \right] + \frac{q_\Pi^2}{2^\lambda} \end{aligned}$$

Game $\mathbf{G}_2$. In this game, the $\beta_i, 1 \leq i \leq q_\Pi$, values are sampled a priori – line 2 of procedure Init of Figure 8.

$\text{Init}(\lambda)$	$\text{Finalize}(\mathbf{m}^*, \sigma^*, \text{id}^*, \text{mpk})$
1 : $(\Pi_{\rightarrow}, \Pi_{\leftarrow}) = (\emptyset, \emptyset)$	1 : $Z := \text{mpk}$
2 : $(\beta_i)_{1 \leq i \leq q_\Pi} \leftarrow \mathbb{B}$	2 : $(u'^*, t'^*, R^*) := \sigma^*$
3 : $z \leftarrow \mathbb{Z}_p^*$	3 : if $u'^* = 0 \cup t'^* = 0 \cup R^* = \mathcal{O}$:
4 : $Z := zG$	4 : abort
5 : return Z	5 : $\alpha^* := \varphi(R^*)$
$\text{bRO}(\alpha)$	6 : $\beta^* := \psi(\alpha^*)$
1 : if $(\alpha, \cdot) \in \Pi$:	7 : $t^* := \psi(\beta^*)$
2 : return $\Pi(\alpha)$	8 : if $t^* = 0$:
3 : $k \leftarrow k + 1$	9 : abort
4 : $\beta \leftarrow \beta_k$	10 : $h^* := \text{H}(\text{id}^*)$
5 : if $(\cdot, \beta) \in \Pi$:	11 : $h'^* := \text{G}(\mathbf{m}^*, \text{id}^*)$
6 : abort	12 : $R'^* := (u'^*)^{-1}(h'^* R^* + t'^*(h^* G + t^* Z))$
7 : $\Pi_{\rightarrow} \leftarrow \Pi_{\rightarrow} \cup \{(\alpha, \beta)\}$	13 : if $R'^* = \mathcal{O}$:
8 : return β	14 : abort
$\text{bRO}^{-1}(\beta)$	15 : $\alpha'^* \leftarrow \varphi(R'^*)$
1 : if $(\cdot, \beta) \in \Pi$:	16 : $\beta'^* \leftarrow \Pi(\alpha'^*)$
2 : return $\Pi^{-1}(\beta)$	17 : if $\psi(\beta'^*) \neq t'^*$
3 : $\alpha \leftarrow \mathbb{A}$	18 : abort
4 : if $\alpha \in \varphi(\mathbb{G}^*)$:	19 : if $(\alpha'^*, \beta'^*) \in \Pi_{\leftarrow}$:
5 : $a \leftarrow \mathbb{Z}_n^*$	20 : abort
6 : $\alpha \leftarrow \varphi(aG)$	21 : return true
7 : if $(\alpha, \cdot) \in \Pi$:	
8 : abort	
9 : $\Pi_{\leftarrow} \leftarrow \Pi_{\leftarrow} \cup \{(\alpha, \beta)\}$	
10 : return α	

Figure 8: Game $\mathbf{G}_2$ and changes corresponding to Game $\mathbf{G}_3$ where during the Init phase the values of β are sampled *a priori* to allow further the transition to a case where the General Forking Lemma can be applied. Lastly, notice that values previously found are resampled during the queries to the bRO^{-1} oracle.

By introducing this change, consequently, the following procedures corresponding to game $\mathbf{G}_2^{\mathcal{F}_0}$ must also happen. Both changes in bRO and bRO^{-1} allow us to argue that the values that follow are sampled accordingly to the same distribution, even though

happening at different moments. Thus, we have:

$$\Pr \left[\mathbf{G}_2^{\mathcal{F}_0} \Rightarrow 1 \right] = \Pr \left[\mathbf{G}_1^{\mathcal{F}_0} \Rightarrow 1 \right]$$

The nature of the forgery $\sigma^* = (u'^*, t'^*, R^*)$, again, is restricted to queries made backwards. Thus allowing to solve the discrete logarithm related to the u^* part of sk_{id^*} by means of the following equation:

$$s'^* R'^* = h'^* R^* + t'^* U \therefore u = \frac{\eta s'^* r'^* - h'^*}{t'^*}, \quad |\eta| = 1$$

Init(λ)	Finalize($\mathbf{m}^*, \sigma^*, \text{id}^*, \text{mpk}$)
1 : $(\Pi_{\rightarrow}, \Pi_{\leftarrow}) = (\emptyset, \emptyset)$	1 : $Z := \text{mpk}$
2 : $\{\beta_i\}_{1 \leq i \leq q_{\Pi}} \leftarrow \mathbb{B}$	2 : $(u'^*, t'^*, R^*) := \sigma^*$
3 : $z \leftarrow \mathbb{Z}_p^* \parallel \text{msk}$	3 : if $u'^* = 0 \cup t'^* = 0 \cup R^* = \mathcal{O}$:
4 : $Z := zG \parallel \text{mpk}$	4 : abort
5 : return Z	5 : $\alpha^* := \varphi(R^*)$
bRO (α)	6 : $\beta^* := \psi(\alpha^*)$
1 : if $(\alpha, \cdot) \in \Pi$:	7 : $t^* := \psi(\beta^*)$
2 : return $\Pi(\alpha)$	8 : if $t^* = 0$:
3 : $k \leftarrow k + 1$	9 : abort
4 : $\beta \leftarrow \beta_k$	10 : $h^* := H(\text{id}^*)$
5 : if $(\cdot, \beta) \in \Pi$:	11 : $h'^* := G(\mathbf{m}^*, \text{id}^*)$
6 : abort	12 : $R'^* := (u'^*)^{-1}(h'^* R^* + t'^*(h^* G + t^* Z))$
7 : $\Pi_{\rightarrow} \leftarrow \Pi_{\rightarrow} \cup \{(\alpha, \beta)\}$	13 : if $R'^* = \mathcal{O}$:
8 : return β	14 : abort
bRO ⁻¹ (β)	15 : $\alpha'^* \leftarrow \varphi(R'^*)$
1 : if $(\cdot, \beta) \in \Pi$:	16 : $\beta'^* \leftarrow \Pi(\alpha'^*)$
2 : return $\Pi^{-1}(\beta)$	17 : if $\psi(\beta'^*) \neq t'^*$
3 : $\alpha \leftarrow \mathbb{A}$	18 : abort
4 : if $\alpha \in \varphi(\mathbb{G}^*)$:	19 : if $(\alpha'^*, \beta'^*) \in \Pi_{\leftarrow}$:
5 : $a \leftarrow \mathbb{Z}_n^*$	20 : abort
6 : $\alpha \leftarrow \varphi(aG)$	21 : for $j \in [k]$:
7 : if $(\alpha, \cdot) \in \Pi$:	22 : if $\beta'^* = \beta'_j$:
8 : abort	23 : return $(j, (\mathbf{m}^*, \text{id}^*, \sigma^*))$
9 : $\Pi_{\leftarrow} \leftarrow \Pi_{\leftarrow} \cup \{(\alpha, \beta)\}$	
10 : return α	

Figure 9: Key only unforgeability notion game for the proof of Theorem 5 transitioning to the semi-logarithm problem, rewinding the bijective random oracle and using the Lemma 1.

Game $\mathbf{G}_3$. By the reasoning previously shown, the event in which the forger succeeds corresponds to the event in which an attacker solves the semi-logarithm problem, thus extracting the user's secret key. This means that the winning condition of Game $\mathbf{G}_3$ relates to Game $\mathbf{G}_2$ in the following manner:

$$\Pr \left[\mathbf{G}_2^{\mathcal{F}_0} \Rightarrow 1 \right] \leq \Pr \left[\mathbf{G}_3^{\mathcal{F}_0} \Rightarrow 1 \right] + \varepsilon_{\text{slp}}.$$

The probability of winning game 3 employing a forward query to the bijective random oracle is given by the acc probability of the General Forking Lemma described in Theorem 1. Thus, we have, by the difference lemma and indistinguishability between games combined:

$$\Pr \left[\mathbf{G}_3^{\mathcal{F}_0} \Rightarrow 1 \right] = \text{acc}, \text{ gfrk} \geq \text{acc} \left(\frac{\text{acc}}{q_\Pi} - \frac{1}{2^\lambda} \right), \Pr \left[\mathbf{G}_3^{\mathcal{F}_0} \Rightarrow 1 \right] \geq \varepsilon - \varepsilon_{\text{slp}} - \frac{q_\Pi^2}{2^\lambda}$$

Thus, applying the General Forking Lemma, we can write $\varepsilon_{\text{slp}} \geq \text{gfrk}$, and we can bound ε by considering two cases:

Case 3. If $\text{acc}/2q_\Pi < 1/2^\lambda \leq \text{acc}/q_\Pi$, then we obtain a result identical to [FKP16] in terms of the advantage of solving the semi-logarithm problem:

$$\varepsilon \leq \varepsilon_{\text{slp}} + q_\Pi (q_\Pi + 2) / 2^\lambda$$

Case 4. If $1/2^\lambda \leq \text{acc}/2q_\Pi$, then:

$$\begin{aligned} \varepsilon_{\text{slp}} &\geq \text{gfrk} \geq \text{acc} \left(\frac{\text{acc}}{q_\Pi} - \frac{1}{2^\lambda} \right) \\ &\geq \text{acc} \left(\frac{\text{acc}}{q_\Pi} - \frac{\text{acc}}{2q_\Pi} \right) \\ &\geq \frac{\text{acc}^2}{2q_\Pi} \therefore \text{acc} \leq \sqrt{2q_\Pi \varepsilon_{\text{slp}}} \Rightarrow \varepsilon \leq \varepsilon_{\text{slp}} + \frac{q_\Pi^2}{2^\lambda} + \sqrt{2q_\Pi \varepsilon_{\text{slp}}} \end{aligned}$$

□